

LAB – 1

1. Tic – Tac – Toe Game

```
import random

def print_board(board):
    print('  1   2   3')
    print(f'1 {board[0][0]} | {board[0][1]} | {board[0][2]}')
    print('  ---+---+---')
    print(f'2 {board[1][0]} | {board[1][1]} | {board[1][2]}')
    print('  ---+---+---')
    print(f'3 {board[2][0]} | {board[2][1]} | {board[2][2]}')

def check_win(board, player):
    for row in board:
        if all(s == player for s in row):
            return True
    for col in range(3):
        if all(board[row][col] == player for row in range(3)):
            return True
    if all(board[i][i] == player for i in range(3)) or \
        all(board[i][2 - i] == player for i in range(3)):
        return True
    return False

def check_draw(board):
    for row in board:
        for spot in row:
            if spot == ' ':
                return False
    return True

def place_marker(board, player, row, col):
    if 0 <= row < 3 and 0 <= col < 3 and board[row][col] == ' ':
        board[row][col] = player
        return True
    return False
```

```

def get_player_choice(player):
    while True:
        try:
            row = int(input(f"Player {player}, enter row (1-3): ")) - 1
            col = int(input(f"Player {player}, enter column (1-3): ")) - 1
            if 0 <= row < 3 and 0 <= col < 3:
                return row, col
            else:
                print("Invalid input. Row and column must be between 1 and 3.")
        except ValueError:
            print("Invalid input. Please enter a number.")

def get_computer_choice(board):
    empty_spots = []
    for r in range(3):
        for c in range(3):
            if board[r][c] == ' ':
                empty_spots.append((r, c))
    return random.choice(empty_spots)

def play_game():
    board = [[' ' for _ in range(3)] for _ in range(3)]
    human_player = 'X'
    computer_player = 'O'
    current_player = human_player
    game_over = False

    print("Welcome to Tic Tac Toe!")
    print(f"You are '{human_player}', the computer is '{computer_player}'.")

    while not game_over:
        print_board(board)

        if current_player == human_player:
            row, col = get_player_choice(current_player)
        else:
            print(f"Computer ({computer_player}) is making a move...")
            row, col = get_computer_choice(board)

```

```

    if place_marker(board, current_player, row, col):
        if check_win(board, current_player):
            print_board(board)
            if current_player == human_player:
                print(f"Player {current_player} wins! Congratulations!")
            else:
                print(f"Computer ({current_player}) wins! Better luck next time!")
            game_over = True
        elif check_draw(board):
            print_board(board)
            print("It's a draw!")
            game_over = True
        else:
            current_player = computer_player if current_player == human_player else human_player
    else:
        if current_player == human_player:
            print("That spot is already taken or invalid. Try again.")

print("Game Over!")

```

OUTPUT:

```

play_game()
... Welcome to Tic Tac Toe!
You are 'X', the computer is 'O'.
  1  2  3
1  |  |
  ---+---+---
2  |  |
  ---+---+---
3  |  |
Player X, enter row (1-3): 2
Player X, enter column (1-3): 2
  1  2  3
1  |  |
  ---+---+---
2  | X |
  ---+---+---
3  |  |
Computer (O) is making a move...
  1  2  3
1  |  |
  ---+---+---
2  | X | O
  ---+---+---
3  |  |
Player X, enter row (1-3): 3
Player X, enter column (1-3): 3
  1  2  3
1  |  |
  ---+---+---
2  | X | O
  ---+---+---
3  |  | X
Computer (O) is making a move...

```

```

  1   2   3
1   | 0 |
  ---+---+---
2   | X | 0
  ---+---+---
3   |   | X
Player X, enter row (1-3): 1
Player X, enter column (1-3):
Invalid input. Please enter a number.
Player X, enter row (1-3): 1
Player X, enter column (1-3): 1
  1   2   3
1 X | 0 |
  ---+---+---
2   | X | 0
  ---+---+---
3   |   | X
Player X wins! Congratulations!
Game Over!
```

2. Vacuum Cleaner Agent

```
import random
DIRTY = 0
CLEAN = 1
class Environment:
    def __init__(self, initial_state_A, initial_state_B):
        self.room_A = initial_state_A
        self.room_B = initial_state_B
        print("Initial Environment State:")
        self.display_state()

    def display_state(self):
        state_A = "Clean" if self.room_A == CLEAN else "Dirty"
        state_B = "Clean" if self.room_B == CLEAN else "Dirty"
        print(f" Room A: {state_A} ({self.room_A})")
        print(f" Room B: {state_B} ({self.room_B})")

    def get_room_state(self, room):
        if room == 'A':
            return self.room_A
        elif room == 'B':
            return self.room_B
        else:
            raise ValueError("Invalid room. Must be 'A' or 'B'.")

    def clean_room(self, room):
        if room == 'A':
            self.room_A = CLEAN
            print(f" Room A cleaned.")
        elif room == 'B':
            self.room_B = CLEAN
            print(f" Room B cleaned.")
        else:
            raise ValueError("Invalid room. Must be 'A' or 'B'.")
```

```

def all_clean(self):
    return self.room_A == CLEAN and self.room_B == CLEAN

class VacuumCleanerAgent:
    def __init__(self, environment, initial_location):
        self.environment = environment
        self.location = initial_location
        print(f"Vacuum Cleaner starts in Room {self.location}.")

    def perceive(self):
        room_state = self.environment.get_room_state(self.location)
        return self.location, room_state

    def act(self, room_location, room_state):
        print(f" Agent is in Room {room_location}. It perceives the room as {"Clean" if room_state == CLEAN else "Dirty"}.")
        if room_state == DIRTY:
            print(f"   Cleaning Room {room_location}...")
            self.environment.clean_room(room_location)
            return 'Clean'
        else:
            if room_location == 'A':
                self.location = 'B'
                print("   Room A is clean, moving to Room B.")
                return 'Move B'
            else:
                self.location = 'A'
                print("   Room B is clean, moving to Room A.")
                return 'Move A'

def run_simulation(steps=10):
    while True:
        state_A_input = input("Enter initial state for Room A (0 for DIRTY, 1 for CLEAN): ")
        try:

```

```

        initial_state_A = int(state_A_input)
        if initial_state_A in [DIRTY, CLEAN]:
            break
        else:
            print("Invalid input. Please enter 0 or 1.")
    except ValueError:
        print("Invalid input. Please enter a number (0 or 1).")
while True:
    state_B_input = input("Enter initial state for Room B (0 for DIRTY, 1 for CLEAN): ")
    try:
        initial_state_B = int(state_B_input)
        if initial_state_B in [DIRTY, CLEAN]:
            break
        else:
            print("Invalid input. Please enter 0 or 1.")
    except ValueError:
        print("Invalid input. Please enter a number (0 or 1).")

env = Environment(initial_state_A, initial_state_B)

while True:
    user_location = input("Enter the vacuum cleaner's starting room (A or B): ").upper()
    if user_location in ['A', 'B']:
        break
    else:
        print("Invalid input. Please enter 'A' or 'B'.")

agent = VacuumCleanerAgent(env, user_location)

for step in range(steps):
    print(f"\n--- Step {step + 1} ---")
    if env.all_clean():
        print("All rooms are clean! Simulation ended.")
        break

```

```

    current_location, current_room_state = agent.perceive()
    agent.act(current_location, current_room_state)

if not env.all_clean():
    print("\nSimulation finished because max steps reached. Not all rooms are clean.")
else:
    print("\nSimulation finished. All rooms are clean.")
env.display_state()

```

OUTPUT :

```

Enter initial state for Room A (0 for DIRTY, 1 for CLEAN): 0
Enter initial state for Room B (0 for DIRTY, 1 for CLEAN): 1
Initial Environment State:
    Room A: Dirty (0)
    Room B: Clean (1)
Enter the vacuum cleaner's starting room (A or B): b
Vacuum Cleaner starts in Room B.

--- Step 1 ---
    Agent is in Room B. It perceives the room as Clean.
    Room B is clean, moving to Room A.

--- Step 2 ---
    Agent is in Room A. It perceives the room as Dirty.
    Cleaning Room A...
    Room A cleaned.

--- Step 3 ---
All rooms are clean! Simulation ended.

Simulation finished. All rooms are clean.
    Room A: Clean (1)
    Room B: Clean (1)

```

